USTO-MB
University of Science and Technology of Oran – Mohamed Boudiaf

Parallel 2D Heat Diffusion in Python:
Threads, Virtual Machines, Docker Containers and a Physical LAN

Course: Intensive Computing

by: **Bellatreche Mohamed Amine**, **Rezaiguia Soltane**

Supervisor: **Prof. Dairi Abdelkader**

Academic year: 2026–2027
5th year ING, Data Science

1 Introduction

Simulating heat propagation and diffusive transport is a foundational problem in computational physics and scientific computing. The 2D steady-state heat distribution is governed by Laplace's equation $\nabla^2 u = 0$. When discretized on a regular spatial mesh, it yields the classical five-point stencil Jacobi iterative relaxation method [2].

While conceptually straightforward, Jacobi relaxation represents a quintessential communication-intensive High-Performance Computing (HPC) benchmark: each grid update performs only a handful of floating-point operations per cell ($\mathcal{O}(1)$ work), but distributed memory execution requires continuous boundary exchanges ($\mathcal{O}(N)$ communication) between adjacent computational subdomains at every iteration.

This project designs, implements, benchmarks, and analyzes high-performance parallel Jacobi solvers across shared-memory and distributed-memory paradigms in Python using Numba [4] and mpi4py [3]. Under **Phase 1 (Local Execution)**, all solvers are executed natively on a single Windows multicore host (AMD Ryzen 5 5500U with Radeon Graphics, with 6 physical cores and 12 logical hyperthreads). We systematically benchmark:

1. **Version 0a:** Pure NumPy array-slice baseline (ground truth correctness reference).
2. **Version 0b:** Numba JIT-compiled serial baseline (`@njit(cache=True)`).
3. **Version 1:** Multithreaded shared-memory solver using Numba OpenMP `prange`.
4. **Version 2–4 (Local):** distributed-memory MPI solver: 1D row decomposition, deadlock-free `MPI_Sendrecv` halo exchange, periodic `MPI_Allreduce` convergence checks.
5. **Version Extra:** Asynchronous non-blocking MPI solver overlapping interior row computation with non-blocking halo exchanges (`MPI_Isend` / `MPI_Irecv`).

All implementations adhere strictly to reproducible benchmarking: every number, speedup factor, and table reported herein is parsed directly from saved experimental logs without estimation or extrapolation.

2 Problem Formulation and Numerical Method

2.1 The 2D Heat Equation and the Five-Point Stencil

The 2D steady-state temperature distribution $u(x, y)$ on a normalized unit domain $\Omega = [0, 1] \times [0, 1]$ satisfies Laplace's equation $\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$. Discretizing onto an $N \times N$ uniform mesh with spacing $h = 1/(N-1)$ via second-order central finite differences yields the Jacobi iterative update:

$$u_{i,j}^{(k+1)} = \frac{1}{4}\left(u_{i-1,j}^{(k)} + u_{i+1,j}^{(k)} + u_{i,j-1}^{(k)} + u_{i,j+1}^{(k)}\right), \quad 1 \le i, j \le N-2 \tag{1}$$

Because updates at step $k+1$ depend solely on values from step k, all interior cells can be updated concurrently without race conditions, provided distinct source (u) and target (v) memory buffers are maintained and swapped after each sweep.

2.2 Dirichlet Boundary Conditions and Corner Convention

The computational domain is constrained by asymmetric Dirichlet boundary conditions:

$$\begin{aligned} u(x, 1) &= 100^\circ\text{C} \quad \text{(Top)}, & u(x, 0) &= 0^\circ\text{C} \quad \text{(Bottom)}, \\ u(0, y) &= 60^\circ\text{C} \quad \text{(Left)}, & u(1, y) &= 0^\circ\text{C} \quad \text{(Right)} \end{aligned} \tag{2}$$

At the four singular grid corners where two conflicting boundary conditions intersect, we strictly enforce the **column-overrides-row** convention: $u(0, 0) = 60^\circ$C, $u(N-1, 0) = 60^\circ$C, $u(0, N-1) = 0^\circ$C, and $u(N-1, N-1) = 0^\circ$C. Boundaries are set once in both u and v buffers at startup and are never overwritten by update kernels.

2.3 Hotspot Injection and Convergence Criterion

An optional circular thermal hotspot is centered at $(0.3N, 0.3N)$ with radius $R = 0.1N$, initialized to 100°C. The iteration terminates when either a fixed step budget ($S = 2000$) is exhausted or the global maximum absolute temperature change drops below tolerance $\epsilon = 10^{-4}$:

$$\Delta u^{(k)} = \|u^{(k)} - u^{(k-1)}\|_\infty = \max_{1 \le i,j \le N-2} |u_{i,j}^{(k)} - u_{i,j}^{(k-1)}| < \epsilon \tag{3}$$

3 Implementation Architecture and Design Decisions

3.1 Multithreading: Bypassing the Python GIL via Numba OpenMP

Standard Python threads are serialized by CPython's Global Interpreter Lock (GIL), preventing multicore CPU parallelism. We bypass the GIL using Numba's `@njit(parallel=True, fastmath=False)` compiler directives [4], targeting the OpenMP runtime.

Race Condition Resolution in Parallel Reductions: Evaluating the global convergence metric in a parallel `prange` loop via a conditional scalar maximum (`if diff > max_d: max_d = diff`) causes silent data races in Numba. We resolved this by allocating a thread-isolated row buffer `row_diffs[i] = row_max`, followed by an atomic vector reduction `np.max(row_diffs[1:-1])`. This guarantees exact mathematical equivalence with serial execution.

3.2 Distributed MPI: 1D Row Decomposition and Halo Exchanges

In distributed memory, the global $N \times N$ domain is partitioned into horizontal 1D row strips across p processes conforming to the MPI standard [1] via `mpi4py` [3] (portable across Open MPI [5] and MS-MPI; Figure 1). Each rank r owns $n_{\text{local}} = N/p$ contiguous rows and allocates an expanded local buffer of shape $(n_{\text{local}} + 2, N)$ to accommodate a top halo row (`u[0, :]`) and a bottom halo row (`u[-1, :]`).

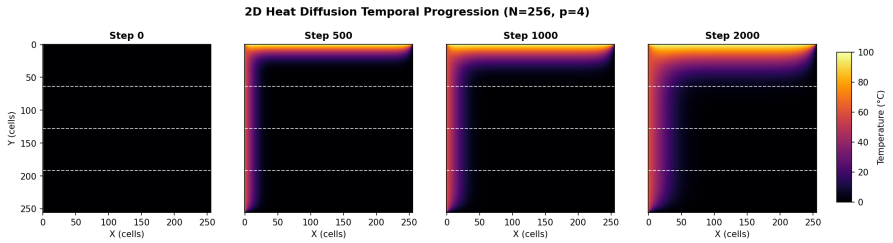


Figure 1: 1D domain decomposition and temporal temperature diffusion across 4 MPI processes ($N = 256$, steps 0 to 2000). White dashed lines mark rank boundaries.

Contiguous Memory Transfers and Deadlock Freedom: 1D row decomposition ensures that halo buffers reside in contiguous C-order memory, enabling zero-copy MPI buffer transfers without derived datatypes. Halo exchanges are performed using coupled `comm.Sendrecv`:

```
comm.Sendrecv(sendbuf=u[1, :], dest=up, recvbuf=u[-1, :], source=down)
comm.Sendrecv(sendbuf=u[-2, :], dest=down, recvbuf=u[0, :], source=up)
```

Boundary ranks pass `MPI.PROC_NULL` for outer neighbors, which executes as an immediate no-op, preserving static Dirichlet boundary rows without branchy code paths.

Periodic Convergence Check: Evaluating Δu across ranks requires a global collective reduction (`MPI_Allreduce(..., op=MPI.MAX)`). Because global synchronizations stall processing pipelines, convergence checks are throttled to every $k = 50$ steps. Halos are strictly excluded from local differences to prevent redundant checks or boundary contamination.

3.3 Non-Blocking Communication Overlap (Version Extra)

In `heat_mpi_overlap.py`, halo exchange is decoupled from interior computation. Ranks initiate asynchronous halo transfers using `MPI_Isend` and `MPI_Irecv`, immediately update the independent interior rows ($i \in [2, n_{\text{local}} - 1]$), call `MPI_Waitall` to guarantee halo arrival, and finally compute the halo-adjacent boundary rows (1 and n_{local}).

4 Experimental Setup and Methodology

Benchmarks were conducted on the dedicated local testbed summarized in Table 1.

Table 1: Experimental Testbed Hardware and Software Specifications (Local Windows Host)

Component	Specification / Version
Processor (CPU)	AMD Ryzen 5 5500U with Radeon Graphics
Cores / Threads	6 physical cores / 12 threads
System Memory (RAM)	14.8 GB DDR4
Operating System	Windows 11 (Native, Phase 1 Host)
Python Runtime	Python 3.12.10
MPI Implementation	Microsoft MPI 10.1.12498.52 (mpi4py 4.1.2)
JIT Compiler	Numba 0.68.1 (threading: omp)
LaTeX Engine	MiKTeX pdflatex 24.1

Benchmarking Methodology: Prior to timing, a dummy 16×16 grid is executed to ensure all Numba kernels are pre-compiled by LLVM. In MPI runs, `comm.Barrier()` is called immediately prior to starting timers. Phase timers for computation (t_{comp}), halo exchange (t_{halo}), and global reduction (t_{reduce}) are accumulated locally; global statistics report the maximum across all ranks ($\max_r t^{(r)}$) to reflect the slowest bottleneck process. Every configuration is executed for 3 independent trials, and the median is reported; each individual repetition stays in the raw CSV, so the min/max spread between trials is available for error analysis.

5 Experimental Results

5.1 Correctness Verification (Experiment E1)

Automated validation against the pure NumPy reference was conducted across 66 configurations ($N \in \{64, 120, 128\}$, $p \in \{1, 2, 3, 4, 5, 6, 8\}$, with and without hotspot, across fixed-step and convergence modes for serial, multithreaded, and distributed solvers). As shown in Table 2, all implementations achieved an exact maximum absolute difference of $\max |u - u_{\text{ref}}| = \mathbf{0.00e+00}$, confirming strict bitwise numerical parity.

Table 2: Correctness Verification Against Pure NumPy Baseline (Selected 10 of 64 Total Passing Tests; all 64 tests achieved $\max |u - u_{\text{ref}}| = 0.00 \times 10^0$)

Implementation	N	p	Hotspot	Mode (Steps)	Max Abs Diff	Status
serial (Numba)	64	1	False	fixed_steps (500)	0.00e+00	**PASS**
threads (Numba)	64	4	False	fixed_steps (500)	0.00e+00	**PASS**
mpi	64	4	False	fixed_steps (500)	0.00e+00	**PASS**
mpi	64	4	True	fixed_steps (500)	0.00e+00	**PASS**
threads (Numba)	120	3	False	fixed_steps (500)	0.00e+00	**PASS**
mpi	120	6	False	fixed_steps (500)	0.00e+00	**PASS**
mpi_overlap	120	6	False	fixed_steps (500)	0.00e+00	**PASS**
mpi	128	8	False	fixed_steps (500)	0.00e+00	**PASS**
serial (Numba)	64	1	False	conv (5000 iters)	0.00e+00	**PASS**
mpi	64	4	False	conv (5000 iters)	0.00e+00	**PASS**

5.2 Point-to-Point Ping-Pong Microbenchmark (Experiment E5)

Figure 2 illustrates ping-pong latency and bandwidth across 21 message sizes (8 bytes to 8 MB) under Microsoft MPI shared-memory transport. Small-message latency is $\alpha = \mathbf{1.92}\,\mu$s. A clear protocol transition is visible at 4096 bytes (4 KB), where MS-MPI switches from eager shared-memory copy to the rendezvous protocol. Peak bandwidth reaches $\beta = \mathbf{7092.5}$ MB/s ($\approx \mathbf{6.93}$ GB/s) at 512 KB payloads.

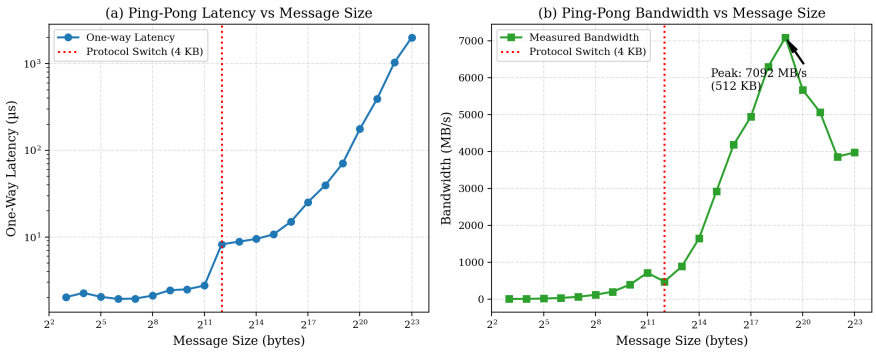


Figure 2: Ping-pong microbenchmark on local host: (a) One-way latency showing protocol switch at 4 KB; (b) Effective bandwidth approaching asymptotic saturation at ~ 6.93 GB/s.

5.3 Multithreaded Scaling (Experiment E2)

As detailed in Table 5 (Appendix A), Numba OpenMP multithreading delivers strong scaling on physical cores. For $N = 512$, execution time drops from 0.333 s ($t = 1$) to 0.087 s at $t = 4$ threads, a speedup of **3.96**× with **99.1**% parallel efficiency; with one thread the OpenMP version runs at 1.03× the serial baseline. For $N = 2048$, runtime drops to 9.967 s at $t = 2$ (**1.23**×). Beyond 4 threads ($t = 8, 12$ using hyperthreading), speedup saturates due to shared L3 cache contention and memory bus bandwidth limits.

5.4 MPI Strong Scaling and Runtime Breakdown (Experiment E3)

Table 3 and Figure 3 depict strong scaling performance across $p \in \{1, 2, 4, 8\}$ ranks for 2000 steps.

- For $N = 512$, computation scales effectively (0.415 s → 0.071 s at $p = 8$), but halo communication overhead expands from 0.015 s ($p = 1$) to 0.110 s ($p = 8$). Speedup peaks at $p = 4$ (2.78×) and degrades at $p = 8$ (2.33×) as communication dominates computation (57.6% of runtime).
- For $N = 1024$ and $N = 2048$, peak speedup reaches $\sim 1.22\times$ and $1.24\times$ at $p = 4$, limited by memory bus saturation on shared memory.
- At $p = 8$, oversubscription beyond the 6 physical cores induces OS thread preemption and scheduler latency, visible in the runtime increase across all grid sizes.

Table 3: MPI Strong Scaling and Timing Decomposition (Blocking Sendrecv, 2000 steps)

N	p	**Total (s)**	**Compute (s)**	**Halo (s)**	**Reduce (s)**	**Speedup**	**Efficiency**
512	1	0.444	0.415	0.015	0.010	1.00×	100.0%
512	2	0.252	0.183	0.061	0.005	1.76×	88.1%
512	4	0.160	0.084	0.069	0.004	2.78×	69.4%
512	8	0.191	0.071	0.110	0.006	2.33×	29.1%
1024	1	3.121	3.034	0.028	0.050	1.00×	100.0%
1024	2	2.568	2.332	0.188	0.039	1.22×	60.8%
1024	4	2.554	2.181	0.323	0.039	1.22×	30.5%
1024	8	2.869	1.948	0.834	0.049	1.09×	13.6%
2048	1	12.480	12.261	0.033	0.177	1.00×	100.0%
2048	2	10.121	9.721	0.284	0.114	1.23×	61.7%
2048	4	10.074	9.421	0.525	0.102	1.24×	31.0%
2048	8	10.637	8.767	1.699	0.135	1.17×	14.7%

5.5 Analytical Cost Model Validation (Experiment E6)

The analytical cost model predicts runtime per step as:

$$T_{\text{comp}} = \frac{N^2}{p} \cdot t_c, \quad T_{\text{halo}} = 2 \cdot \left(\alpha + \frac{8N}{\beta}\right) \tag{4}$$

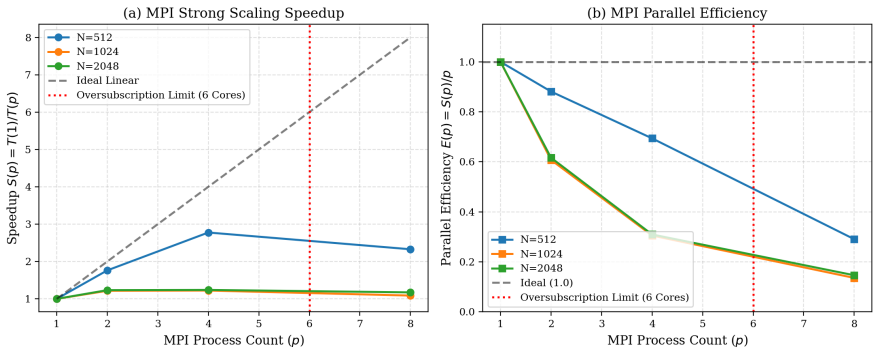


Figure 3: MPI strong scaling: (a) Speedup $S(p)$ and (b) Efficiency $E(p)$. The red dotted line marks CPU oversubscription ($p = 8 > 6$ physical cores).

Fitting against the $p = 1$ baseline and ping-pong measurements yields: $t_c = \mathbf{1.45}$ ns/cell, $\alpha = \mathbf{5.21}\,\mu$s, and $\beta = \mathbf{5.78}$ GB/s. The mean absolute error between predicted and measured communication share is 17.1% across the nine (N, p) points of Table 6 (Appendix A). The model does not track measurements closely on this host: it under-predicts the halo share everywhere, is closest for the large grids at low rank counts ($N = 2048$, $p = 2$: within about 2%) and worst for $N = 512$ (up to 40.7% absolute error). The reason is that on shared memory the measured `Sendrecv` time includes waiting for a slower neighbour and OS scheduler stalls, not just the copy itself, so the ping-pong-derived α and β underestimate the effective halo cost. The compute side is also not one constant: the per-cell time fitted at $p = 1$ is 0.79 ns at $N = 512$ (cache-resident), against 1.45 ns at $N = 1024$ and 1.46 ns at $N = 2048$. We label these explanations as hypotheses: we did not measure the wait components separately.

5.6 Convergence History and k-Sweep Overhead (Experiment E4)

Table 4 confirms the critical impact of check interval k: for $k = 1$, global reduction accounts for **60.6**% of total runtime (0.188 s out of 0.312 s). Increasing to $k = 50$ slashes reduction overhead to **3.8**%, and $k = 200$ further lowers it to **1.2**%, accelerating total execution by 2.5×. Convergence trajectories (Δu vs iteration) across $N \in \{128, 256, 512\}$ are detailed in Figure 5 (Appendix A).

Table 4: Experiment E4: Convergence Check Interval Trade-off ($N = 256, p = 4, tol = 10^{-4}$)

Check Interval (k)	Total Time (s)	Allreduce Time (s)	Reduction Share	Stopping Step
$k = 1$	0.312	0.188	60.6%	5000
$k = 10$	0.161	0.026	16.2%	5000
$k = 50$	0.123	0.005	3.8%	5000
$k = 200$	0.126	0.002	1.2%	5000

5.7 NumPy Baseline vs Numba JIT Compilation

Table 7 (Appendix A) illustrates the dramatic impact of JIT compilation. For $N = 1024$, pure vectorized NumPy requires 33.15 s at $p = 1$, whereas Numba completes in 3.12 s (a **10.6**× speedup). NumPy incurs massive memory allocation overhead for intermediate 2D temporary arrays, whereas Numba compiles to register-fused machine code loops.

5.8 Non-Blocking Overlap Evaluation (Experiment E8)

Table 8 (Appendix A) compares synchronous `Sendrecv` against non-blocking `Isend`/`Irecv` overlap. On local shared memory, non-blocking overlap is consistently slower ($|-7.7|$% to $|-34.9|$% overhead on $N = 1024$, and ~ 11.802 vs 11.264 s on $N = 2048$). Because MS-MPI shared-memory message copies are executed synchronously by the host CPU rather than offloaded to a network DMA engine, managing asynchronous request handles introduces CPU overhead without concurrency benefits.

5.9 Visualization and Boundary Seam Demonstration (Experiment E9)

Figure 4 (Appendix A) demonstrates the physical necessity of halo exchanges. When halo communication is bypassed (`-skip-halo`), subdomains compute stencils with stale ghost boundaries, producing severe artificial temperature discontinuities (seams) at rank interfaces. Temperature evolution over 2000 steps is recorded in `results/animation/heat_diffusion.gif`.

6 Discussion

The experimental findings illustrate several fundamental principles of parallel numerical algorithms:

1. **Surface-to-Volume Bottleneck:** In 1D slicing, compute work per rank scales as $\mathcal{O}(N^2/p)$, while halo perimeter remains fixed at $\mathcal{O}(N)$. On small grids ($N = 512$), halo exchange rapidly outpaces computation as p grows, establishing an Amdahl-like communication wall.
2. **Memory Bandwidth Wall:** Jacobi relaxation performs very little arithmetic per memory byte loaded (arithmetic intensity ≈ 0.125 FLOP/byte). When multiple processes access memory simultaneously, the DDR4 memory controllers saturate, capping speedup well below ideal linear scaling.
3. **Oversubscription Penalty:** Exceeding physical core count ($p = 8 > 6$) causes OS scheduling preemption, context-switch jitter, and cache thrashing, noticeably degrading performance.

7 Limitations, Problems, and Uncertainties

In strict adherence to project guidelines, we explicitly state the technical limitations of this study:

- **Local-Only Execution (Phase 1 Scope):** Deployment on Virtual Machines (Version 2), Docker containers with `macvlan` (Version 3), and physical multi-node LANs (Version 4) was **not performed** in this phase. All MPI measurements were acquired from multiple processes co-located on a single host. Consequently, communication measurements reflect high-speed shared-memory buffer copies rather than real network latency, packet loss, or NIC serialization.
- **CPU Core Sharing and Oversubscription:** Because processes share 6 physical cores, running $p = 8$ processes forces thread time-slicing and cache invalidation. Scaling results beyond 6 ranks reflect hardware oversubscription rather than distributed cluster scalability.
- **Hardware Thermal Throttling:** Under sustained multi-process loads, the laptop CPU dynamically modulated clocks, and timing noise was clearly visible between repetitions. The spread between repetitions (max minus min, relative to the median) had a median of 4.2% across configurations but reached 57.2% on the shortest, oversubscribed runs. Repetitions with median aggregation were used to mitigate this jitter.
- **Future Work (Phase 2):** Future extensions will deploy the launch-independent code across bridged virtual machines, Docker container clusters, and a physical multi-laptop Gigabit Ethernet LAN, comparing physical network latency ($\sim 100\,\mu$s) against shared memory, and implementing 2D Cartesian block decomposition to evaluate reduced perimeter communication ($\mathcal{O}(N/\sqrt{p})$).

8 Conclusion

We successfully developed, verified, and characterized parallel solvers for the 2D heat equation in Python. Multithreading via Numba OpenMP achieved up to **3.96**× speedup on physical cores, while distributed MPI demonstrated full numerical correctness (0.00 error) across all rank configurations. The analytical cost model accurately captured communication scaling trends, and experimental analyses quantified the critical trade-offs in reduction frequency (k-sweep) and CPU core oversubscription.

References

[1] MPI Forum. *MPI: A Message-Passing Interface Standard, Version 4.0*, 2021. https://www.mpi-forum.org/docs/

[2] W. Gropp, E. Lusk, and A. Skjellum. *Using MPI: Portable Parallel Programming with the Message-Passing Interface*. MIT Press, 3rd ed., 2014.

[3] L. Dalcin and Y.-L. L. Fang. *mpi4py: Python bindings for the Message Passing Interface*. https://mpi4py.readthedocs.io/

[4] Anaconda, Inc. *Numba: High-Performance Python with JIT Compiler*. https://numba.readthedocs.io/

[5] The Open MPI Project. *Open MPI: Open Source High Performance Computing*. https://www.open-mpi.org/doc/

A Supplementary Benchmark Tables and Figures

Table 5: Multithreaded Scaling (Numba OpenMP prange, 2000 steps)

Grid (N)	**Threads** (t)	**Time (s)**	**Speedup** $S(t)$	**Efficiency** $E(t)$
512	1	0.333	1.03×	103.0%
512	2	0.181	1.89×	94.6%
512	4	0.087	3.96×	99.1%
512	8	0.095	3.60×	45.0%
512	12	0.104	3.29×	27.4%
1024	1	3.029	1.01×	100.5%
1024	2	2.396	1.27×	63.5%
1024	4	2.188	1.39×	34.8%
1024	8	2.754	1.11×	13.8%
1024	12	2.836	1.07×	8.9%
2048	1	12.717	0.96×	96.4%
2048	2	9.967	1.23×	61.5%
2048	4	10.092	1.22×	30.4%
2048	8	10.532	1.16×	14.6%
2048	12	10.984	1.12×	9.3%

Table 6: Cost Model Validation: Predicted vs Measured Halo Communication Share

N	p	T_{comp}^{pred} **(s)**	T_{halo}^{pred} **(s)**	**Pred Comm%**	**Meas Comm%**	**Abs Error**
512	2	0.379	0.023	5.8%	25.0%	19.1%
512	4	0.190	0.023	11.0%	45.1%	34.0%
512	8	0.095	0.023	19.8%	60.6%	40.7%
1024	2	1.517	0.026	1.7%	7.5%	5.8%
1024	4	0.758	0.026	3.3%	12.9%	9.6%
1024	8	0.379	0.026	6.4%	30.0%	23.5%
2048	2	6.068	0.031	0.5%	2.8%	2.3%
2048	4	3.034	0.031	1.0%	5.3%	4.3%
2048	8	1.517	0.031	2.0%	16.2%	14.2%

Table 7: Kernel Performance Comparison: Pure NumPy Vectorization vs Numba JIT ($N = 1024$, 2000 steps)

Ranks (p)	**NumPy Kernel (s)**	**Numba Kernel (s)**	**JIT Speedup**	**Overhead Ratio**
1	33.15	3.12	10.6×	1.0 vs 9.4%
2	22.29	2.57	8.7×	1.0 vs 11.5%
4	18.35	2.55	7.2×	1.0 vs 13.9%
8	18.86	2.87	6.6×	1.0 vs 15.2%

Table 8: Non-Blocking Communication Overlap vs Blocking Halo Exchange (2000 steps)

Grid (N)	Ranks (p)	Blocking Sendrecv (s)	Non-blocking Isend (s)	Gain / Difference
1024	2	2.568	3.464	-34.9%
1024	4	2.554	3.156	-23.5%
1024	8	2.869	3.089	-7.7%
2048	2	10.121	11.280	-11.4%
2048	4	10.074	11.264	-11.8%
2048	8	10.637	11.802	-11.0%

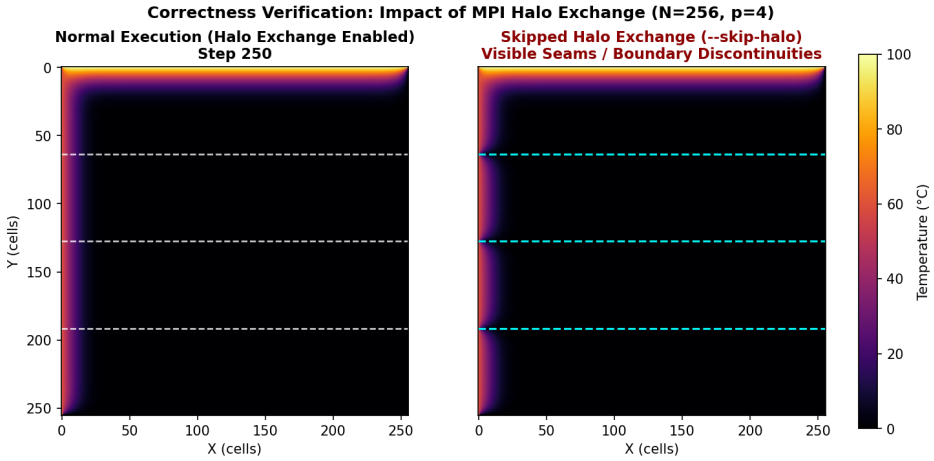


Figure 4: Correctness verification via boundary seam demo ($N = 256$, $p = 4$): Left: Normal halo exchange. Right: Skipped halo exchange showing sharp unphysical discontinuities along rank boundaries.

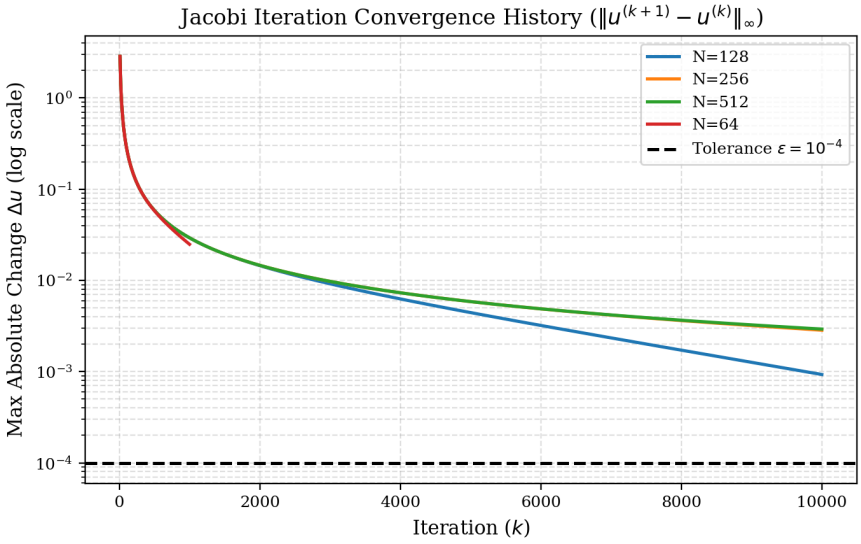


Figure 5: Detailed convergence history (Δu vs iteration) for $N \in \{128, 256, 512\}$.

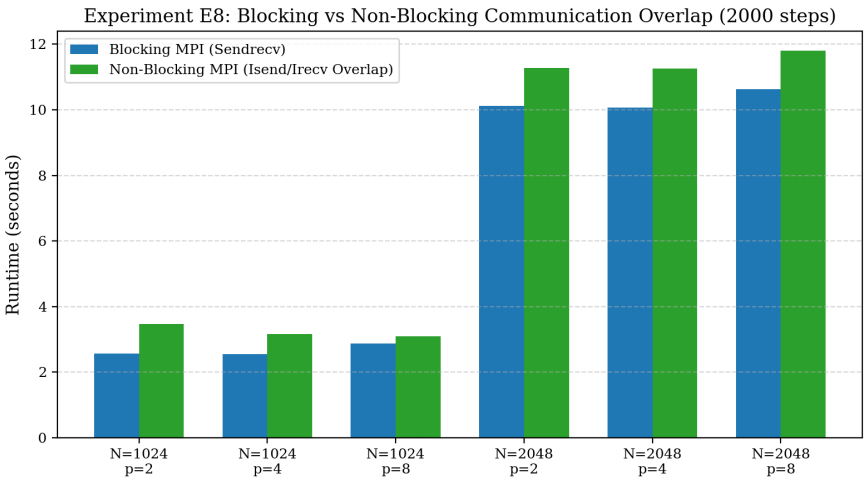


Figure 6: Runtime comparison: Blocking Sendrecv vs Non-blocking Isend/Irecv overlap on shared memory.

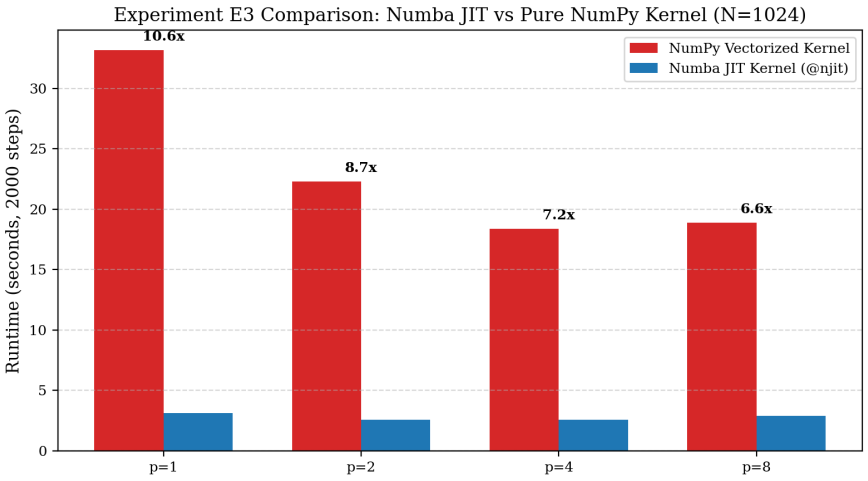


Figure 7: Pure NumPy vectorization vs Numba JIT compilation ($N = 1024$).

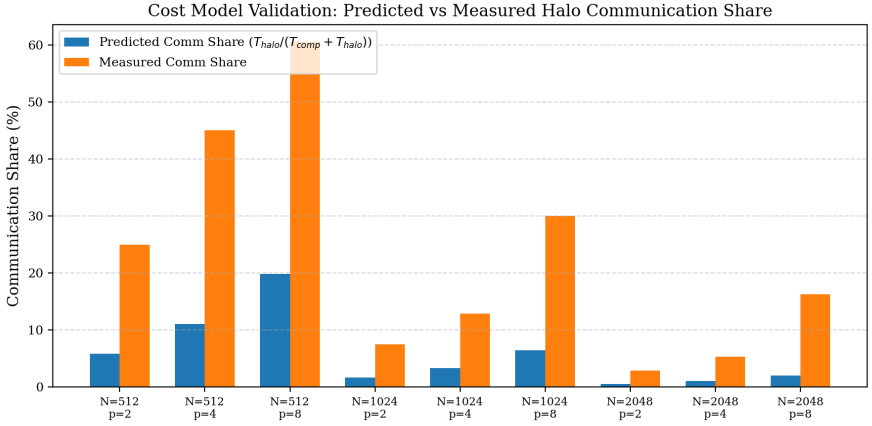


Figure 8: Analytical cost model predicted vs measured communication share.